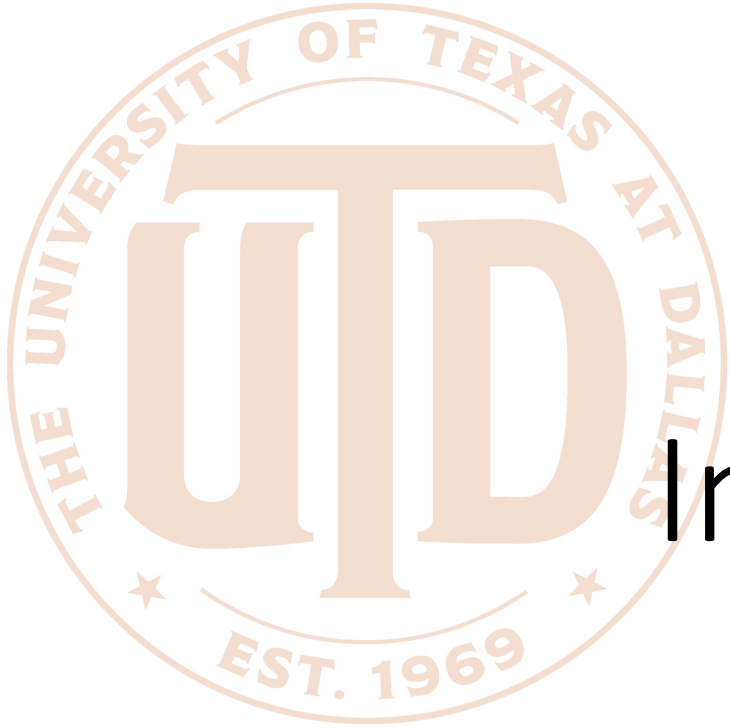




Introduction to ROS

CS 6341 Robotics

Instructor Yu Xiang

The University of Texas at Dallas

Robot Operating System (ROS)

- ROS is a set of software libraries and tools that can be used to build robot applications
 - Drivers, algorithms, developer tools, etc.
- Goal of ROS: support code reuse in robotics research and development
- Operating systems: Unix-based platforms (Ubuntu)
- ROS1 (end of life) and ROS2

<https://www.ros.org/>
<https://wiki.ros.org/>

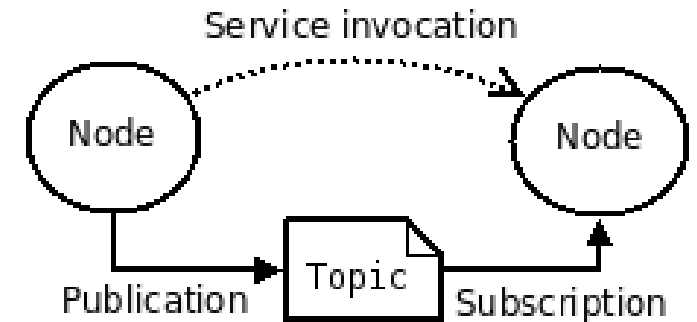
Robot Operating System (ROS)



<https://www.ros.org/>

ROS Computation Graph

- The computation graph is the peer-to-peer network of ROS processes that are processing data together
- Computation graph concepts
 - Nodes: processes that perform computation
 - ROS Master: provides name registration and lookup, nodes can find each other via ROS master
 - Messages: nodes communicate by passing messages, a data structure with type fields (integer, floating, arrays, etc.)



ROS Message Example

File: `sensor_msgs/Image.msg`

Raw Message Definition

```
# This message contains an uncompressed image
# (0, 0) is at top-left corner of image
#
Header header          # Header timestamp should be acquisition time of image
                        # Header frame_id should be optical frame of camera
                        # origin of frame should be optical center of camera
                        # +x should point to the right in the image
                        # +y should point down in the image
                        # +z should point into to plane of the image
                        # If the frame_id here and the frame_id of the CameraInfo
                        # message associated with the image conflict
                        # the behavior is undefined
uint32 height           # image height, that is, number of rows
uint32 width            # image width, that is, number of columns

# The legal values for encoding are in file src/image_encodings.cpp
# If you want to standardize a new string format, join
# ros-users@lists.sourceforge.net and send an email proposing a new encoding.

string encoding         # Encoding of pixels -- channel meaning, ordering, size
                        # taken from the list of strings in include/sensor_msgs/image_encodings.h

uint8 is_bigendian      # is this data bigendian?
uint32 step             # Full row length in bytes
uint8[] data            # actual matrix data, size is (step * rows)
```

std_msgs/Header Message

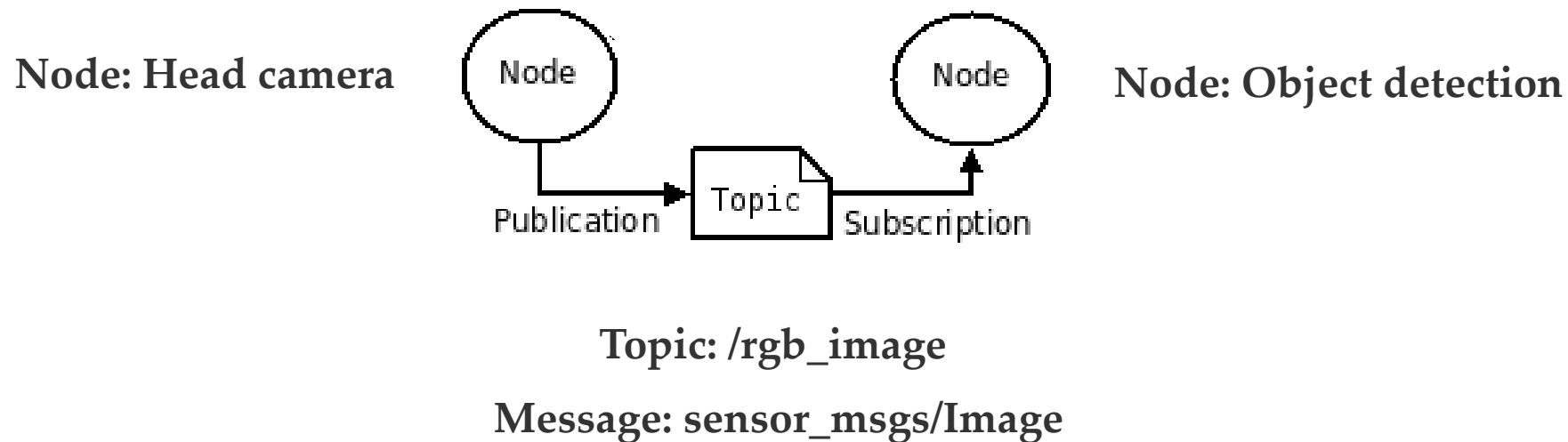
File: `std_msgs/Header.msg`

Raw Message Definition

```
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.
#
# sequence ID: consecutively increasing ID
uint32 seq
#Two-integer timestamp that is expressed as:
# * stamp.sec: seconds (stamp_secs) since epoch (in Python the variable is called 'secs')
# * stamp.nsec: nanoseconds since stamp_secs (in Python the variable is called 'nsecs')
# time-handling sugar is provided by the client library
time stamp
#Frame this data is associated with
string frame_id
```

ROS Computation Graph

- Topics: a node publishes messages to a topic. The topic is the name to identify the content of the message



ROS Computation Graph

- Service: request and reply interactions

Node: Motion Planner



Service: FollowJointTrajectory

Node: Arm Controller

File: `control_msgs/FollowJointTrajectoryGoal.msg`

```
# ===== DO NOT MODIFY! AUTOGENERATED FROM AN ACTION DEFINITION =====
# The joint trajectory to follow
trajectory_msgs/JointTrajectory trajectory

# Tolerances for the trajectory. If the measured joint values fall
# outside the tolerances the trajectory goal is aborted. Any
# tolerances that are not specified (by being omitted or set to 0) are
# set to the defaults for the action server (often taken from the
# parameter server).

# Tolerances applied to the joints as the trajectory is executed. If
# violated, the goal aborts with error_code set to
# PATH_TOLERANCE_VIOLATED.
JointTolerance[] path_tolerance

# To report success, the joints must be within goal_tolerance of the
# final trajectory value. The goal must be achieved by time the
# trajectory ends plus goal_time_tolerance. (goal_time_tolerance
# allows some leeway in time, so that the trajectory goal can still
# succeed even if the joints reach the goal some time after the
# precise end time of the trajectory).
#
# If the joints are not within goal_tolerance after "trajectory finish
# time" + goal_time_tolerance, the goal aborts with error_code set to
# GOAL_TOLERANCE_VIOLATED
JointTolerance[] goal_tolerance
duration goal_time_tolerance
```

File: `control_msgs/FollowJointTrajectoryAction.msg`

Raw Message Definition

```
# ===== DO NOT MODIFY! AUTOGENERATED FROM AN ACTION DEFINITION =====

FollowJointTrajectoryActionGoal action_goal
FollowJointTrajectoryActionResult action_result
FollowJointTrajectoryActionFeedback action_feedback
```

Compact Message Definition

```
control_msgs/FollowJointTrajectoryActionGoal action_goal
control_msgs/FollowJointTrajectoryActionResult action_result
control_msgs/FollowJointTrajectoryActionFeedback action_feedback
```

Using ROS2

- Ubuntu users
 - Install ROS2

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

- Windows and Mac users -> Use docker

Docker: Using ROS2 with Windows and Mac

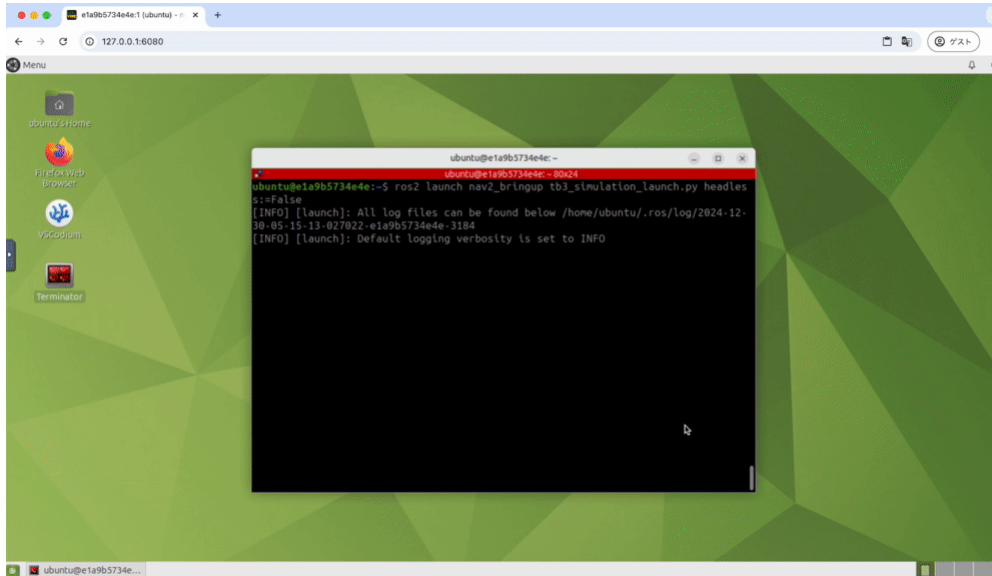
- An open platform that enables you to separate your applications from your infrastructure
- Container
 - A lightweight environment that contains everything to run an application
 - A container is a runnable instance of an image
- Image
 - A read-only template with instructions for creating a docker container

docker-ros2-desktop-vnc

- <https://github.com/Tiryoh/docker-ros2-desktop-vnc>

```
docker run -p 6080:80 --shm-size=512m --name ros2-jazzy -v  
C:\data:/home/ubuntu/data ghcr.io/tiryoh/ros2-desktop-vnc:jazzy
```

Command



Docker image name

Docker image tag

Summary

- ROS
- Docker

Further Reading

- ROS wiki <https://wiki.ros.org/>
- Docker document <https://docs.docker.com/get-started/overview/>
- https://yuxng.github.io/Courses/CS6341Fall2025//CS_6341_Docker_Instructions.pdf